Watermarking

Table of Contents

- Unified Streaming and Forensic A/B Watermarking
  - DASH-IF Demo
- Irdeto TraceMark
  - Live workflow (DASH & HLS)
  - VOD DASH Byte-Range workflow
- Nagra NexGuard
  - Apache SmartEmbedder
  - Using USP and NexGuard ASE
  - Unified Origin configuration

Unified Streaming and Forensic A/B Watermarking

Forensic watermarking provides an efficient deterrent against pirated copies of content.

It is the means by which a unique identifying code is inserted into a media asset. By adding a unique identity disseminated throughout a piece of media, that content, along with its owner, becomes identifiable.

Digital watermarks are used to enforce contractual compliance between a content owner and the intended recipient. It provides proof of misuse and a link back to the source of a leakage. It is one of the most powerful means to protect media content.

If you want to deliver a watermarked stream at scale, the recommended approach is to rely on Origin creating the A/B sets and do the switching at the CDN, perhaps even at the network's edges.

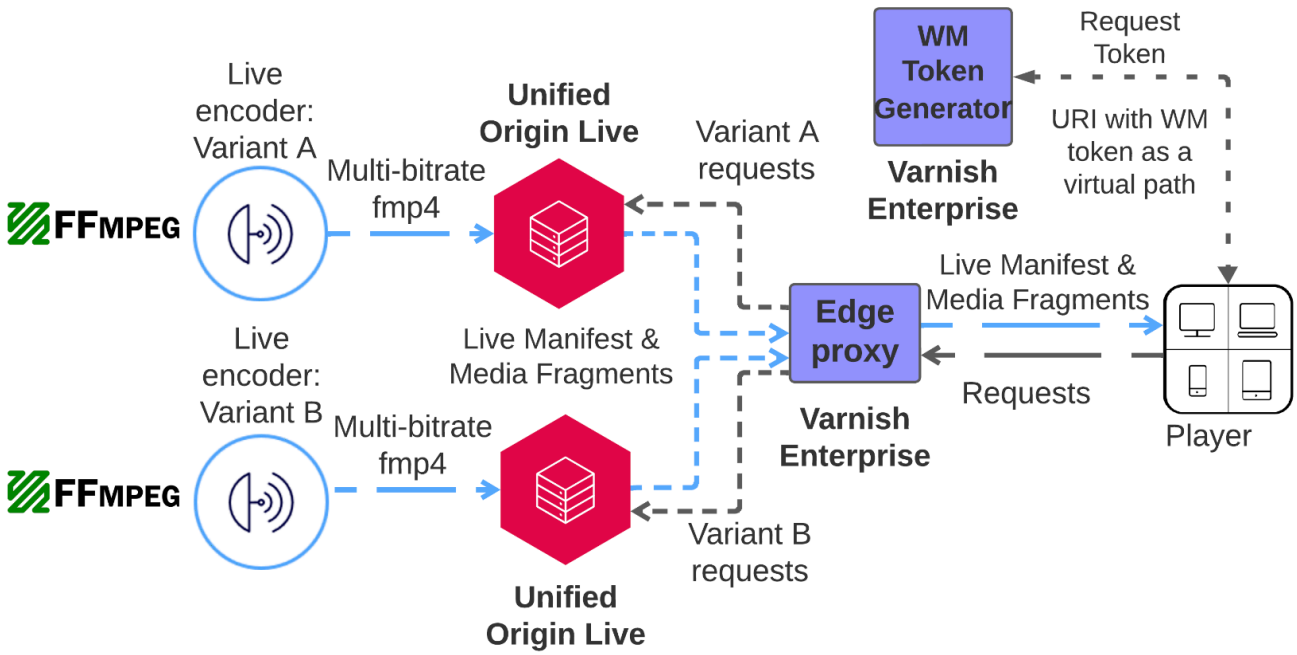
This way, you deliver the same client manifest to all viewers, with those viewers making requests with a unique identifier based on which the watermarking solution will route the request to version A or B of the stream, both of which are hosted by Origin (each using a separate server manifest).

This has two distinct advantages:

- Viewers don't see the watermarking pattern reflected in the client manifest (which they would if the client manifest would have been manipulated by a proxy downstream from Origin but upstream from a CDN)
- Both A and B versions of each segment can be easily cached by the CDN because from the CDN's point of view the A and B versions come from different sources (the A and B versions of the stream hosted by Origin)

DASH-IF Demo

On Github we host a Forensic A/B Watermarking DASH-IF Demo which looks like the following:



This demo was part of the Standards based OTT A/B Watermarking at Scale talk at Mile High Video 22024.

Irdeto TraceMark

With TraceMark, Irdeto provides a watermarking service that can be integrated into a content distribution workflow with *Unified Origin*. The service delivers customers a personalized watermarked video stream in DASH, HLS TS, or HLS fMP4 using segment switching. To learn more about its specifics, please contact Irdeto.

The sections below explain how to configure Origin when setting up Irdeto TraceMark for Live and VOD workflows.

Live workflow (DASH & HLS)

The Live setup requires two publishing points (A and B) and an epoch-locking enabled Encoder that will push uniquely watermarked streams to those publishing points . The encoder uses an epoch-based timecode to generate the segment names in a predictable way and ensures synchronization of such segment names on both publishing points.

For Origin configuration, the output segments must be of the same length and keep the same sequence number as the segments sent by the Live encoder. This is achieved for both DASH and HLS by setting fragment_duration to the exact fraction of the segment length. Additionally, in case of DASH, the SegmentTemplate for the URL must contain the $Time$ variable to include timestamps for identifying the specific requested segment.

Note that the example below doesn't take into account DRM options:

```
#!/bin/bash

mp4split -o live/A/example.isml \
--restart_on_encoder_reconnect \
--hls.no_audio_only \
--hls.client_manifest_version=4 \
--fragment_duration=48/25 \
--mpd.segment_template=time \
--start_segments_with_iframe

mp4split -o live/B/example.isml \
--restart_on_encoder_reconnect \
--hls.no_audio_only \
--hls.client_manifest_version=4 \
--fragment_duration=48/25 \
--mpd.segment_template=time \
--start_segments_with_iframe
```

VOD DASH Byte-Range workflow

Irdeto TraceMark is compatible with DASH-IF Forensic A/B Watermarking standard. Therefore, the VOD setup requires a compatible Encoder and a CDN supporting this standard. The resulting configuration supports only VOD Byte Ranges Based streaming.

A compatible encoder that is able to create A/B versions of fMP4 files watermarks the required assets. The resulting A/B files must have all the fragments byte-aligned and be accompanied with the corresponding CBOR sidecar file. Both versions must be located in separate directories called a and b. Besides, a separate directory called WMPaceInfo must contain the sidecar (.cbor) files. The filenames must be the same across those three directories (see the example at the bottom).

To enable encryption, the following command is run on all renditions of each A/B version of the stream on both a and b directories:

```bash
#!/bin/bash

mp4split -o example_1080.cmfv --cpix=../example.cpix example_1080.mp4
mp4split -o example_2160.cmfv --cpix=../example.cpix example_2160.mp4
mp4split -o example_720.cmfv --cpix=../example.cpix example_720.mp4
```

With encryption enabled, the following command is used to generate a compliant DASH manifest. It is run only once on either a or b directory. The output file is afterwards moved to the parent directory (see the example at the bottom):

```bash
#!/bin/bash

mp4split -o example.mpd \
  --package_mpd --mpd.profile="urn:mpeg:dash:profile:isoff-on-demand:2011" \
  --fragment_duration=2000/1000 \
example_1080.cmfv example_2160.cmfv example_720.cmfv
```

The resulting content must then be served to the CDN from any standard web server or cloud storage with the following structure:

```
├── example.mpd
├── a
│   ├── example_1080.cmfv
│   ├── example_2160.cmfv
│   └── example_720.cmfv
├── b
│   ├── example_1080.cmfv
│   ├── example_2160.cmfv
│   └── example_720.cmfv
└── WMPaceInfo
    ├── example_1080.cbor
    ├── example_2160.cbor
    └── example_720.cbor
```

Nagra NexGuard

Apache SmartEmbedder

The Apache SmartEmbedder (ASE) is a module that allows the integration of the NexGuard-Streaming technology into a content distribution workflow. It is intended to organize per-session water- marking for MP4 file download mode and ABR streaming mode.

It is available on Red Hat 6 & 7 and Ubuntu 14.

For installation and configuration please refer to the NexGuard-Streaming Apache SmartEmbedder documentation.

The input to ASE is content prepared with the NexGuard-Streaming Preprocessor (which includes H.264 encoding) and is stored on disk as a .mmrk preprocessed file. The output from ASE is byte range of watermarked MP4 with a unique ID. The NexGuard video watermark is imperceptible. The solution enables forensic tracking and is a deterrent against content theft.

Watermarked MP4 progressive files (a.k.a. 'mmrk' files) with multiple bit rate renditions can be grouped in a manifest file with the extension 'smil' or 'ism'. Thus, customers can identify which files have been encoded at the same time with different bit rates. Here is an example with 2 bit rate renditions:

```xml
<?xml version="1.0" encoding="utf-8"?>
<smil xmlns="http://www.w3.org/2001/SMIL20/Language">
  <body>
    <switch>
      <video src="myFile__750kbps.mp4" systemBitrate="750000"/>
      <video src="myFile_1500kbps.mp4" systemBitrate="1500000"/>
    </switch>
  </body>
</smil>
```

Using USP and NexGuard ASE

The Apache server is started and its home page is accessible from any web browser:

```
http://ase-hostname
```

At least one 'mmrk' file has been copied in the Apache server DocumentRoot (sub)folder (e.g. `/var/www/html/myFile_750kbps.mmrk`):

```
http://ase-hostname/streampath.txid.manifest-ext
```

The path consists of the following elements:

Value	Description
streampath	stream path without the extension (e.g. 'myFolder/myStream-1000Kbps')
txid	payload value to smart embed (in hexadecimal)
manifest-ext	manifest extension ('smil' or 'ism')

Unified Origin configuration

The *Unified Origin* is configured following Dynamic Manifests, so using the `IsmProxyPass` directive, e.g.:

```
<Directory /var/www/ase>
  IsmProxyPass http://ase-hostname/
</Directory>
```

The IsmProxyPass directive will map a request containing 'ase' in it's patch to the apache-smart-embedder. There should be no 'ase' directory on disk.

Url Description

The URL for accessing a SmartEmbedded stream is different from the standard Unified Streaming URL since a transaction ID must be specified:

```
http://usp-hostname/ase/strname.txID.ext/post-fix
```

Value	Description
strname	stream name without extension (folder and stream path, e.g. myFolder/myStream)
txID	unique transaction identifier for each client (hexadecimal value)
ext	media extension ('mp4' or 'smil')
post-fix	option information specific to player technology (e.g. 'Manifest' for Smooth Streaming, '.m3u

Single bit rate

For single bit rate, the media type use is 'mp4'. The myID parameter is a per-session identifier as a hexadecimal number without the `0x` prefix.

Value	Description
HLS	http://usp-hostname/ase/myStream.A1B2.mp4/.m3u8
Smooth Streaming	http://usp-hostname/ase/myStream.mp4/Manifest

From the received manifest file, then any chunk (or fragment) can be downloaded, for instance HLS:

```
http://usp-hostname/ase/myStream.A1B2.mp4/myStream-400k-video=407000-89.ts
```

Adaptive bit rate

For adaptive bit rate, the media type use is 'ism'.

Value	Description
HLS	http://usp-hostname/myStream.A1B2.ism/.m3u8
Smooth Streaming	http://usp-hostname/myStream.A1B2.ism/Manifest

From the received manifest file, then any chunk (or fragment) can be downloaded, for instance for HLS:

```
http://usp-hostname/ase/myStream.A1B2.ism/myStream-400k-video=407000-89.ts
```